Software Testing, Quality Assurance and Maintenance	Winter 2026

Lecture 20 — March 20, 2026

Patrick Lam *version 1*

Termination

Dafny does prove termination for loops and recursion. It does so by proving that there is something that decreases and that is bounded. Usually Dafny manages to guess what is decreasing. When that is something over natural numbers, the lower bound is often 0. You can hover over the loop to see the inferred decreases clause. Sometimes Dafny can't guess, and you have to provide an annotation.

```
method Decreases()
{
  var i := 20;
  while 0 < i
    invariant 0 ≤ i
    decreases i // could be inferred
    {
      i := i − 1;
    }
}
```

Note that the thing that is decreasing might be the negation of something that's increasing.

```
method Decreases2()
{
  var i, n := 0, 20;
  while i < n
    invariant 0 ≤ i ≤ n
    decreases n − i
    {
      i := i + 1;
    }
}
```

Dafny can guess this one too, from the loop condition $i < n$. The lower bound is from $i \leq n$ in the invariant, which implies $n - i \geq 0$. The bound doesn't have to be constant, as in binary search.

Exercise 10. In the loop above, the invariant $i \leq n$ and the negation of the loop guard allow us to conclude $i == n$ after the loop, as we've previously checked with an assert. If the loop guard was instead $i \neq n$ (as in Exercise 7), then the negation of the guard immediately gives $i == n$, without needing the loop invariant. Change the loop guard to $i \neq n$ and delete the invariant. What happens?

We have also seen the recursive function fib. Dafny also guesses the decreases annotation here. We could specify it explicitly:

```
function fib(n:nat): nat
  decreases n
{
  if n = 0 then 0
```

```
  else if n = 1 then 1
  else fib(n−1) + fib(n−2)
}
```

There is a longer Dafny tutorial on termination, and we'll probably go through that one as well.

Arrays

In Dafny we have one-dimensional arrays array$\langle T \rangle$ for any T (though here we'll only consider array$\langle int \rangle$), as well as array?$\langle T \rangle$ which includes possibly-null arrays.

Array access is as you might expect, $a[i]$, and there is a built-in length field: a.Length. Part of Dafny's compile-time verification includes showing that all array accesses are in-bounds, i.e. betweeen 0 and Length $-$ 1 inclusive.

We saw array allocation right at the beginning of this document:

```
    var arr := new int[2];
```

Let's express the easy part of the contract for the Find method.

```
method FindPartialContract(a: array<int>, key: int) returns (index: int)
  // partial contract
  ensures 0 ≤ index ⟹ index < a.Length ∧ a[index] = key
{
  // can you write code that satisfies this postcondition?
  // it can be done with one statement.
}
```

This is the case where the key is in the array. It's at index index. The conjunct index $< a$.Length guards against out-of-bounds array accesses that might be caused by a[index], and works because Dafny has short-circuit logical ands/ors (like C and many other languages).

For the other case, we need to say that key does not exist at *any* index in the array, so we need quantifiers.

Quantifiers. Dafny supports quantifiers in expressions. Use a double-colon (::) after the variable name; it shows up as a dark · in Emacs and VS Code.

```
method WithQuantifier()
{
  assert ∀ k • k < k + 1;
}
```

(This verifies, but there's a warning about triggers. We won't talk about this here.)

Typically we quantify over some more limited set than "all integers" (Dafny infers the type of k, though you can specify it if you want). For instance, all integers that are valid indices into an array, i.e. between 0 and a.Length.

```
  ∀ k • 0 ≤ k ≤ a.Length ⇒ a[k] ≠ key
```

We can thus write the whole contract:

```
method Find(a: array<int>, key: int) returns (index: int)
  ensures 0 ≤ index ⟹ index < a.Length ∧ a[index] = key
  ensures index < 0 ⟹ ∀ k • 0 ≤ k < a.Length ⟹ a[k] ≠ key
{
    // TODO implement this
}
```

Linear search is the easiest way to implement Find:

```
  index := 0;
  while index < a.Length {
    if a[index] = key {
      return;
    }
    index := index + 1;
  }
  index := −1;
```

and when we plug that into the contract, we see that Dafny can't prove the postcondition saying that if index is negative, then key is not in the array. Makes sense: all we know upon leaving the loop is that index has reached the end of the array. We need to add an invariant showing that the part of the array we've visited so far does not contain key.

```
  invariant ∀ k • 0 ≤ k < index ⟹ a[k] ≠ key
```

Dafny complains: it doesn't know that index doesn't go off the end of the array. This invariant works.

```
  invariant 0 ≤ index ≤ a.Length
```

We often see loop invariants where we build the property that we need one element at a time, as we do here. In this case, the property is that the visited part of the array doesn't contain the key that we're looking for.

Exercise 11. Write a method that takes an integer array, which it requires to have at least one element, and returns an index to the maximum of the array's elements. Annotate the method with pre- and postconditions that state the intent of the method, and annotate its body with a loop invariant to verify it.

```
method FindMax(a: array<int>) returns (index: int)
  // TODO put preconditions and postconditions here
{
  // TODO implement this
}
```

Linear search is fine, but if the array is sorted, then binary search is faster. We can't prove that binary search is correct unless we can somehow express a property about the array being sorted. Which we can. But let's see if we can refactor this property, using predicates.

Predicates. Dafny has functions, which can return anything. Dafny also has predicates, which return booleans. We are going to define the sorted predicate which will return true if and only iff its array argument is sorted. Using this predicate makes our expressions shorter and more readable (as long as the name of the predicate corresponds to its meaning).

```
predicate sorted(a: array<int>)
{
```

```
  ∀ j, k • 0 ≤ j < k < a.Length ⟹ a[j] ≤ a[k]
}
```

A predicate is like a function, but the return type must be boolean, and so you don't have to write this.

Dafny won't compile this, and says that there is an insufficient reads clause.

Framing. Dafny needs to know when heap locations might change.

We know that functions and predicates can't change the heap. But Dafny isn't a purely functional language—methods can change the heap.

So, Dafny wants to know when the value of sorted might change between calls. In a world where there are two arrays, and a method writes to one of the arrays, a reads annotation tells Dafny *which* array sorted is reading from, and whether that array might have been changed by the method. If not, then the array has the same sortedness before and after the method was called.

```
predicate sorted(a: array<int>)
  reads a
{
  ∀ j, k • 0 ≤ j < k < a.Length ⟹ a[j] ≤ a[k]
}
```

The reads annotation specifies a set of memory locations that the function is allowed to access. Here, it is an array, which means that sorted can access all of the elements of the array. It could also be object fields and sets of objects. We'll probably check that later.

As we've seen, Dafny checks that the reads clause specifies everything that the function might read. This includes any functions that the function transitively calls.

Conversely, as we've seen, methods can read anything they want. But they must declare any changes to memory using a modifies clause.

These clauses allow Dafny to reason about methods and functions in isolation, and hence they make verification much more tractable.

Dafny only requires framing information for memory on the heap, which includes arrays and objects, but not sets, sequences, or multisets (which we'll talk about later).

Exercise 12. Create a sortedAndDistinct predicate that returns true exactly when the array is sorted and all its elements are distinct.

Exercise 13. Create a sortedAndNonNull predicate that accepts references to a possibly-null array and that returns true exactly when the array is sorted and not null.

Binary Search. Predicates: software engineering principles for annotations. Here we use the sorted predicate in the precondition for BinarySearch. Dafny thus knows that the array is sorted when verifying the method body.

```
method BinarySearch(a: array<int>, value: int) returns (index: int)
  requires 0 ≤ a.Length ∧ sorted(a)
```

```
  ensures 0 ≤ index ⟹ index < a.Length ∧ a[index] = value
  ensures index < 0 ⟹ ∀ k • 0 ≤ k < a.Length ⟹ a[k] ≠ value
{
    //...
}
```

Here's an implementation of binary search.

```
  var low, high := 0, a.Length;
  while low < high
    invariant 0 ≤ low ≤ high ≤ a.Length
    invariant ∀ i •
      0 ≤ i < a.Length ∧ ¬(low ≤ i < high) ⟹ a[i] ≠ value
    {
      var mid := (low + high) / 2;
      if a[mid] < value {
        low := mid + 1;
      } else if value < a[mid] {
        high := mid;
      } else {
        return mid;
      }
    }
    return −1;
```

Let's look at the implementation and the invariants.

```
    invariant 0 ≤ low ≤ high ≤ a.Length
    invariant ∀ i •
      0 ≤ i < a.Length ∧ ¬(low ≤ i < high) ⟹ a[i] ≠ value
```

The first invariant says that the search is looking between $[\mathsf{low}, \mathsf{high})$, and the second invariant says that the searched-for value is not anywhere outside that range.

Reasoning about how the first invariant continues to hold is straightforward.

For the second invariant, Dafny needs to show that changing low or high doesn't miss the value that we are searching for. It can use the fact that predicate sorted holds. Taking the first if case, we are bumping up low to above mid, based on sortedness and the fact that $\mathsf{a}[\mathsf{mid}] < \mathsf{value}$, which ensure that the value isn't in that part of the range.

Off-by-one errors are easy to write. Dafny catches them in this case.

Exercise 14. Change the assignments in the body of BinarySearch to set low to mid or to set high to $\mathsf{mid} - 1$. What goes wrong?

Conclusion

We've seen a bunch of the major features of Dafny and used it to verify some simple code, up to a binary search implementation. There are more features: objects, sequences and sets, data structures, lemmas, modules, etc. We'll probably talk about them in the next few lectures. You don't really need them to do the assignment this year. Maybe next year.

There is a lot of information about Dafny online, including other tutorials and the reference.

SE 465 is not actually the course on specifications. That's SE 463. But we use formal specifications in this part of SE 465 to prove that implementations conform to these specifications, and that they are using other code consistently with that code's specifications. Both tests and verification are tools that you can use to verify that your code does what it's supposed to do. Tests are more practical, while verification is conceptually more powerful.

We'll work through some more examples from the stqam-1261-dafny repo, which you can find at `https://git.uwaterloo.ca/stqam-1261/dafny`.

First, consider the greatest common divisor. We will take the textual specification to be this:

```
/*
  Define a recursive function, called gcd, that computes a GCD of two numbers.
  Use the following properties of gcd.
  (a) gcd(n, n) = n
  (b) gcd(m, n) = gcd(m-n, n) whenever m > n
  (c) gcd(m, n) = gcd(m, n-m) whenever n > m
*/
```

We can then write this specification as a (recursive) Dafny function:

```
function gcd(m: nat, n: nat): nat
  requires m > 0 ∧ n > 0
{
  if (m = n) then m
  else if (m > n) then gcd(m − n, n)
  else gcd(m, n − m)
}
```

For our purposes, we take this to be the definition of gcd. I don't think Dafny can prove that the declarative definition of gcd is equal to this recursive definition. Since Dafny's natural numbers start at 0, we specifically define gcd to start at 1 to ensure termination: we are subtracting numbers and we need to subtract at least 1 to $m + n$ on each recursive call if we are to make progress.

The contract of GcdCalc can be as follows:

```
method GcdCalc(m: nat, n: nat) returns (res: nat)
  requires m > 0 ∧ n > 0;
  ensures res = gcd(m, n)
```

That is, we use the gcd function we've defined. And we mirror the precondition of the function.

The implementation is straightforward:

```
{
  var n1, m1 := n, m;
  while (n1 ≠ m1)
  {
    if (m1 < n1) {
     n1 := n1 − m1;
    } else {
     m1 := m1 − n1;
    }
  }
  assert(m1 = n1);
  return n1;
}
```

We don't absolutely need the final assert, but it also doesn't hurt to put it there as documentation. What we do need, though, is an invariant. Without an invariant, Dafny doesn't know what to make of the loop. So, here, you can read along, and agree with what I'm writing, but really, you need

to practice writing some invariants to be able to actually do this. Go do some invariant writing. More than on the assignment.

Anyway, the sensible first invariant to put in is mirroring the precondition. (Indeed, Dafny complains that it can't show the function precondition for function gcd with no precondition). We need both m and n to be positive.

```
invariant m > 0 ∧ n > 0
```

That does help, but Dafny still can't prove termination, nor can it prove the postcondition. Of course. We don't know anything about what the loop is actually computing at this point, just that m and n are strictly positive.

A good guess for the additional invariant is

```
invariant gcd(m, n) = gcd(m1, n1)
```

I don't really know how to give you intuition about this. The invariant has to somehow gcd and you should have some idea that this gcd algorithm is preserving the gcd. For you, it can be based on something you vaguely remember from MATH 135.

Under the hood with Dafny

Some of the things that Dafny does are a form of syntactic sugar which increase its usability. Let's look at this method again and desugar it, using what we know about Hoare Logic.

```
method add_by_inc(n: nat, m: nat) returns (r: nat)
  requires true
  ensures r = m + n
{
  r := n;
  var i := 0;
  while i < m
    invariant i ≤ m
    invariant r = n + i
  {
    i := i + 1;
    r := r + 1;
  }
  return r;
}
```

Dafny converts this into a bunch of verification conditions which it passes along to z3. So, first, let's convert the requires and ensures clauses into assumes/asserts. Also, to make our life easier, we'll combine the two invariants.

```
method add_by_inc(n: nat, m: nat) returns (r: nat)
{
  assume {:axiom} true;
  r := n;
  var i := 0;
  while i < m
    invariant i ≤ m ∧ r = n + i
  {
    i := i + 1;
    r := r + 1;
  }
  assert r = m + n;
```

```
  return r;
}
```

Remember that we had to show that the invariant held upon entry, that it was preserved by the loop; and we exit the loop with the invariant and not the loop condition. Let's do that manually here. First, that the invariant holds upon entry.

```
method add_by_inc(n: nat, m: nat) returns (r: nat)
{
  assume {:axiom} true;
  r := n;
  var i := 0;
  assert i ≤ m ∧ r = n + i;
  while i < m
    invariant i ≤ m ∧ r = n + i
  {
    i := i + 1;
    r := r + 1;
  }
  assert r = m + n;
  return r;
}
```

Next, we want to show that the invariant holds at the end of the loop body, if we know nothing but the invariant upon entry to the loop (at least about i and r). We stop the verification after the end of the loop body by putting assume false; no paths escape that assumption, so the verifier just does nothing further there.

```
method add_by_inc(n: nat, m: nat) returns (r: nat)
{
  assume {:axiom} true;
  r := n;
  var i := 0;
  assert i ≤ m ∧ r = n + i;
  // havoc i, r;
  i, r := *, *;
  assume {:axiom} i ≤ m ∧ r = n + i
  if i < m
  {
    i := i + 1;
    r := r + 1;
    assert i ≤ m ∧ r = n + i
    assume {:axiom} false;
  }
  assert r = m + n;
  return r;
}
```

Let's think about what we've done here. In fact, we've made Dafny check the three properties of inductive loop invariants.

1. Clearly, the first assert checks that it holds upon entry to the loop.
2. The havoc and assume erase knowledge of everything (havoc) but the loop invariant (assume) before the body of the loop executes. The loop body executes if the loop condition is true (and Dafny tracks the fact that the loop condition is true), and keeps the loop invariant. Then, Dafny checks that the loop invariant is preserved by the loop body. That's all that it checks for the loop body; it must not propagate facts further.
3. Finally, the do-nothing else branch of the if takes the invariant once again, and conjoins it with the negation of the loop condition, and continues from there; Dafny goes on and checks

what happens after the loop exits—in particular, whether the postcondition exiting the loop is strong enough.

There was also an inferred decreases clause. We can use $\mathsf{m} - \mathsf{i}$ as the thing that decreases. It's easy to desugar that. We're going to use ghost variables; ghost means that a variable is specification-only—it doesn't need to be included in the executable. Nothing bad happens if you make them normal variables, but it's not as tidy.

We check that $\mathsf{m} - \mathsf{i}$ is smaller at the end of the loop than it was at the beginning of the loop, and also that it is nonnegative (the integers have infinite decreasing chains while the naturals don't; no infinite decreasing chain and strict decreasing together imply termination).

```
// as before..
{
  ghost var rank0 := m - i;
  i := i + 1;
  r := r + 1;
  ghost var rank1 := m - i;
  assert rank1 < rank0;
  assert rank1 ≥ 0;
  assert i ≤ m ∧ r = n + i
  assume {:axiom} false;
}
```

You can use this desugaring to help prove things. When Dafny is complaining about something, you can explicitly put in the assertions that it's trying to show, look at the code, and understand why the assertion fails. You can also put other assertions to figure out what else Dafny knows. And you can put in assumes to see which case Dafny can't show (we'll talk more about that).

In the dafny repository, you can find the above example as well as one with nested loops (file vcc.sol.dfy).

Fast exponentiation

This example is in fast_exp.prob.dfy and solved in fast_exp.sol.dfy in the Dafny repository. We will work through how we specify and verify the fast exponentiation algorithm, which uses repeated squaring to compute exponents; it uses the fact that

$$e^{2n} = (e^n)^2$$

i.e. to compute e^{2n} you can compute e^n and then square the result.

First, let's define a function exp. This part is straightforward; nothing to see here. The comments in the provided Dafny file provide the definition of exp that we're working from. This is a recursive definition.

```
function exp(x: real, n:nat) : real
{
  if n = 0 then 1.0
  else x * exp(x, n-1)
}
```

Next, we'd like to provide a specification for the provided implementation. Let's first look at the implementation and convince ourselves that it works.

```
method exp_by_sqr (x0: real, n0: nat) returns (r: real)
{
  if (n0 = 0) { return 1.0; }
  if (x0 = 0.0) { return 0.0; }
  var x, n, y := x0, n0, 1.0;
  while (n > 1)
  {
    if (n % 2 = 0)
    {
      x := x * x;
      n := n / 2;
    }
    else
    {
      y := x * y;
      x := x * x;
      n := (n-1)/2;
    }
  }

  return x * y;
}
```

We put aside the base cases, which are clear. In the even case, we see that we are repeatedly squaring x and halving n, which is exactly in line with the identity we saw above. In the odd case, we now have a variable y, which stores extra xs. Here, let's look.

$$x^{2n+1} = x \times x^{2n}$$

So we're assigning $x \times y$ to y, i.e. storing an extra factor of x here. And, in the end, we return x times the leftover ys that we've stored.

Dafny now has test cases, so let's also run a few test cases to convince ourselves that this implementation works. It can be useful to do that before we try to prove something that turns out to be not true. Instead of assert, which is statically verified, we use expect to write what we expect (shout-out to Kieran for finding this; the documentation on Dafny tests is hard to find.)

```
method {:test} TestExpBaseCases()
{
  var res1 := exp_by_sqr(17.0, 0);
  expect res1 = 1.0;
  var res2 := exp_by_sqr(2.2, 1);
  expect res2 = 2.2;
  var res3 := exp_by_sqr(0.0, 11);
  expect res3 = 0.0;
}

method {:test} TestExpEven()
{
  var res1 := exp_by_sqr(1.0, 4);
  expect res1 = 1.0;
  var res2 := exp_by_sqr(2.0, 4);
  expect res2 = 16.0;
  var res3 := exp_by_sqr(3.0, 2);
  expect res3 = 9.0;
}
```

```
method {:test} TestExpOdd()
{
  var res1 := exp_by_sqr(1.0, 5);
  expect res1 = 1.0;
  var res2 := exp_by_sqr(2.0, 5);
  expect res2 = 32.0;
  var res3 := exp_by_sqr(3.0, 3);
  expect res3 = 27.0;
}
```

We can run these test cases. Here I run them on a version of the program that I've verified, but you can also run tests on not-yet-verified code if you annotate relevant methods with {:verify false} and call dafny test --allow-warnings --no-verify.

```
$ dafny test fast_exp.dfy

Dafny program verifier finished with 9 verified, 0 errors
TestExpBaseCases: PASSED
TestExpEven: PASSED
TestExpOdd: PASSED
```

What's the specification? Well, we are returning r and it had better be equal to exp(x0, n0), so here we are:

```
ensures r = exp(x0, n0)
```

Onwards to verification. As always, we at least need loop invariants (if not more). What could they be? Once again, we look at the postcondition, which essentially says x * y == exp(x0, n0). Then, Dafny gives two errors: (1) this invariant is not strong enough to prove the postcondition; and (2) the invariant is also not maintained by the loop. So now we have to debug verification issues.

It seems like it's a better bet to make sure the invariant is strong enough—might as well do that before we try to make it go through. Even if we expect it to be true, let's explicitly write out what we have upon exit to the loop, i.e. the negation of the loop condition plus the invariant. We can also explicitly write the postcondition as an assertion.

```
assert n ≤ 1 ∧ exp(x0, n0) = exp(x, n) * y;
assert x * y = exp(x0, n0)
```

Dafny can of course prove the first assertion but not the second one.

If we look at that first assertion, and we remember that Dafny starts its nat type at 0, then we might have $n = 0$. Is that indeed the case that is causing trouble? We can add an assume n == 1 to see. If we do that, then we see that Dafny is fine with the postcondition. Or, we can add assert n == 1 and see that Dafny can't prove that. We don't ever actually want to have $n = 0$ here, and we can realize that the loop invariant did not constrain n. This reminds us to add the usual kind of invariant on the loop variable:

```
invariant 1 ≤ n ≤ n0
```

and we can see that Dafny then only complains about the invariant being maintained by the loop body, but not about the postcondition.

Recall that what Dafny is effectively doing is putting an assume at the beginning of the loop body and an assert at the end. We can be explicit and put these, and indeed, Dafny complains at the end.

In the loop body, there is the n even case and the n odd case. We can focus on just one case by putting assume false on the other case, so that Dafny doesn't go on and try to verify that the invariant is maintained on that case. Let's handle the even case and put the assume on the odd case. (If we put it on both cases, then Dafny thinks it can verify the thing.) We can move our assert to the then case as well.

```
    assume exp(x0, n0) = exp(x, n) * y;
    if (n % 2 = 0)
```

```
    {
      x := x * x;
      n := n / 2;
      assert exp(x0, n0) = exp(x, n) * y;
    }
    else
    {
      assume false;
      y := x * y;
      x := x * x;
      n := (n-1)/2;
    }
```

What about at the beginning of the then-clause? Of course we can put the loop invariant there. That will work, but it doesn't get us anywhere. But, do we know that exp(x0, n0) == exp(x*x, n/2) * y? We should! (This is what the loop body does). If we assert that, then Dafny says no. What if we move the assume into the body of the then-clause? Dafny still says no.

We can simplify this even more and just assert

```
  assert exp(x, n) = exp(x*x, n/2);
```

and see that Dafny doesn't know about the key property of the exp function that we started this verification with.

We can go and remove extra asserts and assumes; we continue the verification of exp_by_sqr with just one assume:

```
  if n % 2 = 0 {
    assume exp(x, n) = exp(x*x, n/2)
    //...
```

We can do the same process on the else branch to get the assume:

```
  } else {
    assume exp(x, n) = exp(x*x, (n-1)/2) * x;
    //...
```

Using lemmas. Our proof so far has some assumes, which mean that we haven't quite proven correctness yet. We can look at the assume and say, well, of course it's true, but we don't really know. I'm going to deus ex machina some lemmas which we are going to use. Lemmas are functions that tell Dafny some additional facts about their parameters.

```
lemma exp_even (x: real, n: nat)
  requires n % 2 = 0
  ensures exp (x, n) = exp (x*x, n/2)
{
  // needs some code here rather than assume false
  assume false;
}

lemma exp_odd(x: real, n: nat)
  requires n % 2 = 1
  ensures exp (x, n) = exp (x*x, (n-1)/2)*x
{
  // needs some code here rather than assume false
  assume false;
}
```

Let's look at exp_even. If we call it from some other Dafny code, then, just like with a function, the precondition has to hold (i.e. n has to be even). Upon return from this lemma, we know the postcondition—the fact that exp (x, n) == exp (x*x, n/2). Which is just the property that we started with.

Lemmas are ghost code and don't get compiled into the executable. They exist only for their effects on the proof. Dafny takes them into account when it's verifying code. You also have to prove that the postcondition is actually true, but we will first use the lemma and then prove it. Here's what that looks like.

```
if (n % 2 = 0)
{
  exp_even(x,n);
  x := x * x;
  n := n / 2;
}
else
{
  exp_odd(x,n);
  y := x * y;
  x := x * x;
  n := (n-1)/2;
}
```

The precondition of the lemmas holds because it is just the immediately-preceding if condition. Dafny can therefore know that the postcondition is true. It happens that these postconditions are exactly the same as the assumes that we had previously inserted, so Dafny can verify the whole exp_by_sqr method (modulo the still-missing proofs of the lemmas).

Proving lemmas. Before we prove the lemmas for exp_by_sqr, we can learn how to prove some simpler lemmas. The file lemmas.prob.inclass.dfy contains some example lemmas which we'll work through.

We usually need lemmas when we have some function which Dafny doesn't know the properties of. Dafny is pretty good with properties of its + operator, but let's provide a add function which does the same thing. Dafny doesn't know that function.

```
function add (x: nat, y: nat): nat
{
  if (y = 0) then x
  else add (x, y-1) + 1
}

/* Lemma: x + 0 = 0 */
lemma add_zero_lemma (x: nat)
  ensures add (x, 0) = x
{
  // REPLACE BY PROOF
  assume(false);
}
```

It turns out that Dafny is actually smart enough to prove add_zero_lemma on its own, but let's provide our own proof of it, as a pedagogical exercise.

Dafny supports calc blocks, where we put a sequence of lines that are each equal to the next one. Using these blocks, we can supply calculational proofs.

```
lemma add_zero_lemma (x: nat)
```

```
  ensures add (x, 0) = x
{
  calc {
    add(x,0);
    x;
  }
}
```

This calc block shows that add(x,0) == x, which we knew already. If we make it wrong on purpose (e.g. add(x,1) == x), then Dafny complains.

This next lemma is actually harder to prove—it doesn't follow immediately from the definition of add. Dafny is still smart enough to prove it automatically, but we'll do it manually again.

```
lemma zero_add_lemma (x: nat)
  ensures add (0, x) = x
{
  assume(false);
}
```

We can split the proof into cases, e.g.

```
lemma zero_add_lemma (x: nat)
  ensures add (0, x) = x
{
  if x = 0 {
  } else if x = 1 {
  } else {
    assume false;
  }
}
```

Dafny verifies this, so we know that it can handle the 0 and 1 cases automatically. (These are add(0,0) == 0 and add(0, 1) == 1 respectively). Let's write a calc block for the other cases.

```
  calc {
    add(0,x);
    add(0, x-1)+1;
  }
```

What we really want to do now is to invoke induction, using 0 and 1 as the base cases. We can write the inductive hypothesis as an assume for calc:

```
  calc {
    add(0,x);
    add(0, x-1)+1;
    {zero_add_lemma(x-1);}
    (x-1)+1;
    x;
  }
```

Dafny checks that the preconditions to zero_add_lemma are met. Since there are none, those hold. Also, because it is a recursive call inside the braces, Dafny checks that the recursion is well-founded: the parameter x-1 is smaller than x, and there are base cases. We can then use the postcondition to replace add(0, x-1) with (x-1) in the calc block, and the rest of the calculation is easy.

There are a few more lemmas that Dafny can prove automatically. It's good to try to prove them manually as well, and the solutions are in lemmas.sol.dfy. I will put this one here, though.

```
lemma one_plus_add (x: nat, y: nat)
  ensures add(x, y) + 1 = add (x+1, y)
{ }
```

Here is one that Dafny cannot prove automatically.

```
/** Lemma: (x + y) = (y + x)
*/
lemma add_comm_lemma (x: nat, y: nat)
  ensures add(x, y) = add(y, x)
{
}
```

We get the usual error that there is a postcondition that could not be proved.

We can try some base cases:

```
/** Lemma: (x + y) = (y + x) */
lemma add_comm_lemma (x: nat, y: nat)
  ensures add(x, y) = add(y, x)
{
  if x = 0 { }
  else if y = 0 { }
  else {
  }
}
```

and see that those cases verify, but not the general case. Time to put a calc block.

```
  calc {
    add(x, y);
    add(x, y-1) + 1; // from the definition of add
```

This is straightforward so far. Then, we want to use induction again. The obvious thing to use induction on is what we have on the previous line, arguments (x, y-1). And we can then deploy the postcondition of the present lemma. Note that if you get the inductive arguments wrong, Dafny will indeed complain.

```
    {add_comm_lemma(x,y-1);}
    add(y-1,x) + 1;
```

Dafny can actually do it pretty much automatically from here, but let's explicitly also bring the +1 back into the add() parameters using induction again. The remaining steps after that are straightforward. We'll be pedantic about -1+1, though Dafny doesn't need that either.

```
    {one_plus_add(y-1,x);}
    add(y-1 + 1, x);
    add(y,x);
  }
```

Proving fast_exp lemmas. You don't actually need to use calc to prove these—you can iterate on n and write some invariants—but the solutions in fast_exp.sol.dfy do use calc and mutual recursion, and I guess that's nicer? You decide.

```
lemma exp_even (x:real, n:nat)
  requires n % 2 = 0
  ensures exp (x, n) = exp (x*x, n/2)
{
  if (n = 0) { }
  else
  {
    calc
    {
      exp (x, n);
      x * exp (x, n-1);
```

```
      { exp_odd (x, n-1); }
      x * x * exp (x*x, (n-2)/2);
      exp (x*x, n/2);
    }
  }
}

lemma exp_odd(x: real, n: nat)
  requires n % 2 = 1
  ensures exp (x, n) = exp (x*x, (n-1)/2)*x
{
  if (n = 0) { }
  else
  {
    calc
    {
      exp (x, n);
      x * exp (x, n-1);
      { exp_even (x, n-1); }
      x * exp (x*x, (n-1)/2);
    }
  }
}
```

Here are my iterative proofs. exp_odd just calls exp_even since, if n is odd, then n-1 is even; and I didn't use exp_odd in my exp_even proof.

```
lemma exp_even (x: real, n: nat)
  requires n % 2 = 0
  ensures exp (x, n) = exp (x*x, n/2)
{
  var i := 0;
  var r, rr := 1.0, 1.0;
  while i < n/2
    invariant 0 ≤ i ≤ n/2
    invariant r * r = rr
    invariant exp(x, i) = r
    invariant exp(x*x, i) = rr
    invariant exp(x, i*2) = rr
  {
    r := r * x;
    rr := rr * x * x;
    i := i + 1;
  }
}

lemma exp_odd(x: real, n: nat)
  requires n % 2 = 1
  ensures (exp (x, n) = exp (x*x, (n-1)/2)*x)
{
  exp_even(x, n-1);
}
```

Bubble Sort

We are now going to verify a bubble sort implementation, which will be helpful for doing the assignment. I am assuming that everyone here has seen bubble sort at some point in the past. It's been longer for me than it's been for you. (Though I wouldn't expect bubble sort to be in CS 341). And, spoiler, there is going to be a twist in the verification.

You can find the bubble sort implementation at bubble_sort.prob.dfy in the Dafny repo.

Auxiliary predicates: sorted and pivot. First, we are going to define a predicate sorted. We've done similar things before. This time we specify a from and to range to check for sortedness. The predicate returns true iff its array parameter a is sorted between the range [from, to).

```
predicate sorted(a: array<int>, from: int, to: int)
  requires 0 ≤ from ∧ to ≤ a.Length
  reads a
{
  ∀ j, k • from ≤ j < k < to ⟹ a[j] ≤ a[k]
}
```

Next, I'm going to introduce a pivot predicate. I'm going to do this before a reminder of how bubble sort works. You may remember pivots being useful, in general, for sorting algorithms.

pivot takes three arguments: an array, a to argument, and a potential pvt. This predicate checks the sub-array [0, to) of its array for whether pvt is a pivot—that is, all elements with index less than pvt are less than (or equal to) all elements of the sub-array with index greater than pvt.

We can encode the predicate as follows:

```
predicate pivot(a: array<int>, to: int, pvt: int)
  requires 0 ≤ pvt ≤ to ≤ a.Length
  reads a
{
  ∀ u, v • 0 ≤ u < pvt < v < to ⟹ a[u] ≤ a[v]
}
```

So, we let u take all values in $[0, \mathsf{pvt})$; and v is in $[\mathsf{pvt}+1, \mathsf{to})$. The predicate is true iff all a[u] are less than or equal to all a[v].

Dafny doesn't complain about this definition, but how do we know that the words correspond to the formalism?

We don't.

We can look at the formalism closely. It seems to be OK, but that is not a guarantee.

Another thing we can do is to run some test cases. I've included both assert and expect checks. In this case, in the Dafny model, probably assert is better: it is possible to verify these statically. For a method, all we check is the postcondition. But this is a function and so we are running the body when we use assert.

```
method {:test} TestPivot()
{
  var singleton := new int[] [4];
  assert pivot(singleton, singleton.Length, 0) = true;
  var three := new int[] [2,3,4];
  expect pivot(three, three.Length, 1) = true;
  var threeB := new int[] [4,3,2];
  expect pivot(threeB, threeB.Length, 1) = false;
  var a := new int[] [3, 5, 9, 2, 11, 13, 15, 12];
  expect pivot(a, a.Length, 0) = true;
  // a[3] < a[1], so false:
  expect pivot(a, a.Length, 2) = false;
  expect pivot(a, a.Length, 3) = true;
}
```

Dafny statically verifies the assert and, when you run the test case, it passes.

Implementing bubble sort. Here is an implementation of bubble sort. Note that it modifies array a in place.

```
method bubbleSort (a: array<int>)
{
  var i:nat := 1;

  while (i < a.Length)
  {
    var j:nat := i;
    while (j > 0)
    {
      if (a[j-1] > a[j]) {
        // Swap a[j-1] and a[j]
        var temp := a[j-1];
        a[j] := temp;
        a[j-1] := a[j];
      }
      j := j - 1;
    }
    i := i+1;
  }
}
```

Let's now review a bit how this works. We know that bubble sort is $O(n^2)$. The outer loop ensures that the subarray from $[0, i)$ is sorted at each iteration. The inner loop makes this happen by bubbling down items inside $[0, i)$ that are out-of-order, starting at index i and pushing them as far as needed, possibly to index 0.

For instance:

```
9, 3, 5, 2
3, 9, 5, 2
3, 5, 9, 2
3, 5, 2, 9
3, 2, 5, 9
2, 3, 5, 9
```

In the lecture notes I'm not showing the loop iterations explicitly, but it's a worthwhile exercise to write the array contents at each stage, and in particular, to note which sub-arrays are sorted.

Let's write a contract. This is perhaps an obvious contract for this method.

```
  requires a.Length > 0
  ensures sorted(a, 0, a.Length)
  modifies a
```

And we know that we have to write invariants for the while loop. We've done this several times already, so we know that this invariant is likely to work:

```
  invariant i ≤ a.Length
  invariant sorted(a, 0, i)
```

As we might expect, because we haven't provided any invariants at all for the inner loop, all Dafny knows coming out of the inner loop is that $j = 0$, which of course does not imply that any sub-array is sorted.

What I did next was look at the expanded loop iterations to see which sub-arrays were sorted. One pattern that is possible to see is that, of course, the sub-array $[0, i)$ always remains sorted in the inner loop; and also, sub-array $[j, i + 1)$ also remains sorted. So we can write this inner loop invariant:

```
  invariant 0 ≤ j ≤ i ≤ a.Length
  invariant sorted(a, 0, i)
  invariant sorted(a, j, i+1)
```

and Dafny doesn't complain.

It might seem like we're done. That was easy?

Are you sure? Let's write a test case.

```
method {:test} TestSort()
{
  var a := new int[] [3, 5, 9, 2, 11, 13, 15, 12];
  bubbleSort(a);
  var b := new int[] [2, 3, 5, 9, 11, 12, 13, 15];
  expect a[..] = b[..];
}
```

Dafny's arrays, like many other languages, are object-like. It's like Java. Comparing == of two arrays is probably not what you want, and definitely not here. We want something like Java's equals(), where we check the contents of the arrays. To do that in Dafny, we can form sequences (a Dafny built-in type) from the arrays, and compare those.

If we run this test case, Dafny reports that it fails. (Since we are calling a method, this really has to be expect and not assert). Why is this? The test harness output is not so helpful, but we can add print a[..] to the test case, printing out the sequence:

```
Dafny program verifier finished with 10 verified, 0 errors
TestPivot: PASSED
TestSort: [3, 5, 9, 9, 11, 13, 15, 15]FAILED
  bubble_sort.prob.dfy(88,1): expectation violation
TestSort2: PASSED
[Program halted] bubble_sort.prob.dfy(12,0): Test failures occurred: see above.
```

Uh oh. That's not what we expected. Somehow there are two 9s and two 15s in the array.

Multisets to the rescue. We have seen that the output is wrong. Even though it verifies. The array is indeed sorted, but it's not at all equal to a permutation of the original array. Our ensures clause is too weak, and forgot to specify that part of the relationship between the old array and the new array. Indeed, a function could just return the empty array and still satisfy the specification.

Permutations are actually not so obvious in Dafny. But Dafny has primitive type multiset, which stores elements and their multiplicities. If the output array is sorted, and if converting both arrays to multisets and comparing them succeeds, then we can say that we have a sort implementation. Let's do that.

```
method bubbleSort2 (a: array<int>)
  requires a.Length > 0
  ensures sorted(a, 0, a.Length)
  ensures multiset(a[..]) = multiset(old(a[..]))
  modifies a
```

Here, old refers to the contents of a upon entry to the method. Dafny has to be able to reason about those while verifying, so that we can compare old and new.

Of course, bubbleSort2 doesn't verify. It shouldn't! We add the multiset constraint as an invariant to both loops as well. Now, Dafny says that the multiset invariant is not preserved by the inner loop. That's odd... the contents are changing in the inner loop? Oh! It turns out that someone put wrong code to swap array elements. There was a bug. Despite verification (of the wrong contract). Let's fix that.

```
var temp := a[j-1];
a[j-1] := a[j];
a[j] := temp;
```

Now Dafny doesn't complain about the multiset invariant anymore. It does complain about how sortedness from 0 to i is not maintained by the loop.

I did mention the pivot predicate a while ago. And the first sorted call from 0 to i seems somewhat fishy. It would make more sense to have that call range from 0 to j, which is the loop variable here. And it would also make sense for a[j] to be the pivot of the sub-array between 0 and $i + 1$. So, this invariant actually works for the inner loop:

```
    invariant 0 ≤ j ≤ i ≤ a.Length
    invariant sorted(a, 0, j)
    invariant pivot(a, i+1, j)
    invariant sorted(a, j, i+1)
    invariant multiset(a[..]) = multiset(old(a[..]))
```

That's all the insight I can give you about where the corrected invariants come from.

Dafny does verify this! It would be smart to still be a bit suspicious. But we can cross-check the verification with a test case.

```
method {:test} TestSort2()
{
  var a := new int[] [3, 5, 9, 2, 11, 13, 15, 12];
  bubbleSort2(a);
  var b := new int[] [2, 3, 5, 9, 11, 12, 13, 15];
  expect multiset(a[..]) = multiset(b[..]);
}
```

which passes. Between the test case (which is not super strong evidence) and the verification, we can be more sure that this implementation is correct, though again, there are still no guarantees.